

Correction TD3

Exercice 1 : classe Produit

```
#include <iostream>

using namespace std;

class Produit {
private:
    int refe;    // Référence du produit
    float puht; // Prix unitaire hors taxes
    int quantite; // Stock disponible
public:
    // Prototypes des méthodes
    void initialise(int r, float p, int q);
    float prixttc();
    float prix_total();
    void retirer(int q);
    void ajouter(int q);
    void afficher();
};

// Méthode d'initialisation
void Produit::initialise(int r, float p, int q) {
    refe = r;
    puht = p;
    quantite = q;
}

// Méthode pour calculer le prix TTC
float Produit::prixttc() {
    return puht * 1.2;
}
```

```

// Méthode pour calculer le prix total en stock
float Produit::prix_total() {
    return quantite * prix TTC();
}

// Méthode pour retirer une quantité du stock
void Produit::retirer(int q) {
    if (q > quantite) {
        cout << "Stock insuffisant ! Vous ne pouvez pas retirer " << q << " unités.\n";
    } else {
        quantite -= q;
        cout << q << " unités retirées. Stock restant : " << quantite << "\n";
    }
}

// Méthode pour ajouter une quantité au stock
void Produit::ajouter(int q) {
    quantite += q;
    cout << q << " unités ajoutées. Nouveau stock : " << quantite << "\n";
}

// Méthode pour afficher les détails du produit
void Produit::afficher() {
    cout << "Produit (Ref: " << ref << ")\n"
        << "Prix HT: " << puht << " EUR, Prix TTC: " << prix TTC() << " EUR\n"
        << "Quantité en stock : " << quantite << "\n"
        << "Valeur totale en stock : " << prix_total() << " EUR\n";
}

// Programme principal
int main() {
    Produit p1;

```

```
// Initialisation de l'objet Produit
p1.initialise(101, 50.0, 20);
// Affichage des informations initiales
p1.afficher();
// Ajout et retrait de stock
p1.ajouter(10);
p1.retirer(5);
p1.retirer(30); // Test du stock insuffisant
// Affichage final
p1.afficher();
return 0;
}
```

Exercice 1 : classe CompteBancaire

```
#include <iostream>
using namespace std;
class CompteBancaire {
private:
    int numero;
    float solde;
public:
    // Prototypes des méthodes
    void initialise(int num, float montant);
    void deposer(float montant);
    void retirer(float montant);
    void virement(CompteBancaire &dest, float montant);
    void afficher();
};
```

```

// Méthode d'initialisation
void CompteBancaire::initialise(int num, float montant) {
    numero = num;
    solde = montant;
}

// Déposer de l'argent
void CompteBancaire::deposer(float montant) {
    solde += montant;

    cout << "Depot de " << montant << " MAD effectue. Nouveau solde : " << solde << "
MAD\n";
}

// Retirer de l'argent
void CompteBancaire::retirer(float montant) {
    if (solde >= montant) {
        solde -= montant;

        cout << "Retrait de " << montant << " MAD effectue. Nouveau solde : " << solde << "
MAD\n";
    } else {
        cout << "Fonds insuffisants ! Retrait refuse.\n";
    }
}

// Effectuer un virement vers un autre compte
void CompteBancaire::virement(CompteBancaire &dest, float montant) {
    if (solde >= montant) {
        solde -= montant;
        dest.solde += montant;

        cout << "Virement de " << montant << " MAD effectue vers le compte N: " <<
dest.numero << ".\n";
    } else {
        cout << "Fonds insuffisants pour le virement !\n";
    }
}

```

```

    }
}
// Afficher les informations du compte
void CompteBancaire::afficher() {
    cout << "Compte N: " << numero << " | Solde : " << solde << "MAD\n";
}
// Programme principal
int main() {
    CompteBancaire compte1, compte2;
    // Initialisation des comptes
    compte1.initialise(101, 500.0);
    compte2.initialise(102, 300.0);
    // Affichage des comptes
    compte1.afficher();
    compte2.afficher();
    // Dépôt et retrait
    compte1.deposer(200);
    compte1.retirer(100);
    // Virement de compte1 à compte2
    compte1.virement(compte2, 250);
    // Affichage final
    compte1.afficher();
    compte2.afficher();
    return 0;
}

```